

# Java OS + JVM — Memorization Sheet

## Q1 — What Happens When Running ``java -jar app.jar``

- Terminal (bash/zsh) is already a running process.
  - Terminal calls OS (fork + exec) to create a new process.
  - OS creates native process and allocates virtual memory, heap, stack, file descriptors.
  - JVM runtime initializes inside that process.
  - JVM loads bytecode and starts `main()`.
  - Mental Model: OS Process → JVM → Your App.
- 

## Q2 — Threads Before ``main()`` Runs

- Multiple threads exist before `main()` executes.
  - OS bootstrap native thread exists.
  - JVM internal threads created during initialization.
  - Examples: GC threads, JIT compiler threads, VM Thread, Reference Handler, Signal Dispatcher.
  - Thread count depends on JVM, GC algorithm, CPU cores, configuration.
- 

## Q3 — Production Thread Debugging

- Classify threads into: Application / Framework / JVM Internal.
- Application threads: `ExecutorService`, `@Async`, custom pools.
- Framework threads: Tomcat `http-nio-*`, Netty `nioEventLoopGroup-*`, `ForkJoinPool`.
- JVM internal threads: GC Thread, VM Thread, `CompilerThread`, Signal Dispatcher.
- Thread name gives hint; stack trace gives truth.
- High CPU debugging: Check `RUNNABLE`, `BLOCKED` (lock contention), `WAITING` (pool starvation).